

SymPy Bug Report

1 Bug 5: Wrong Negative-Real Branch in an Antiderivative

1.1 Minimal Reproducer

```
from sympy import N, diff, integrate, simplify, sqrt, symbols

x = symbols("x")
integrand = sqrt(x**2 + 1)/x
F = integrate(integrand, x)
residual = diff(F, x) - integrand

print("antiderivative:", F)
print("integrand at -2:", integrand.subs(x, -2))
print("derivative at -2:", diff(F, x).subs(x, -2))
print("residual at -2:", simplify(residual.subs(x, -2)))
print("numeric residual:", N(residual.subs(x, -2), 50))
```

1.2 Observed Behavior

```
antiderivative: x/sqrt(1 + x**(-2)) - asinh(1/x) + 1/(x*sqrt(1 + x**(-2)))
integrand at -2: -sqrt(5)/2
derivative at -2: sqrt(5)/2
residual at -2: sqrt(5)
numeric residual: 2.2360679774997896964091736687312762354406183596115
```

SymPy returns an expression as an antiderivative of

$$\frac{\sqrt{x^2 + 1}}{x}.$$

However, differentiating that returned expression and evaluating at $x = -2$ gives $\sqrt{5}/2$, while the original integrand evaluates to $-\sqrt{5}/2$. The residual is therefore $\sqrt{5}$, so this is not a harmless formatting difference or an omitted additive constant.

1.3 Expected Behavior

The derivative of an indefinite integral result should equal the integrand on each interval where the integrand is defined. In particular, for

$$f(x) = \frac{\sqrt{x^2 + 1}}{x},$$

any antiderivative returned for the real variable x must differentiate back to $f(x)$ on both $(-\infty, 0)$ and $(0, \infty)$, up to the usual additive constants on those intervals. At $x = -2$, the derivative should be $-\sqrt{5}/2$, not $+\sqrt{5}/2$.

1.4 Mathematical Explanation

The sign error comes from replacing a branch-sensitive expression by its positive-real branch. For real nonzero x ,

$$\frac{\sqrt{x^2 + 1}}{x} = \text{sign}(x)\sqrt{1 + x^{-2}}.$$

Thus

$$\frac{\sqrt{x^2 + 1}}{x} = \sqrt{1 + x^{-2}} \quad \text{only when } x > 0,$$

whereas on the negative real axis it equals

$$-\sqrt{1 + x^{-2}}.$$

SymPy's returned expression differentiates to the positive branch $\sqrt{1 + x^{-2}}$. That derivative agrees with the integrand for positive real x , but has the wrong sign for negative real x . Since an additive constant disappears under differentiation, no choice of integration constant can repair a pointwise derivative residual of $\sqrt{5}$ at $x = -2$.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant integration path as the Meijer-G indefinite integration fallback. The public call `integrate( $\sqrt{x^2 + 1}/x, x$ )` reaches `Integral.doit` and `Integral._eval_integral` in `sympy/integrals/integrals.py`. The Risch, heuristic Risch, and manual integration paths do not produce an answer. The fallback calls `meijerint_indefinite` near `sympy/integrals/integrals.py:1105`. The result is then produced in `sympy/integrals/meijerint.py` by the internal helper `_meijerint_indefinite_1`. The diagnosis localizes the branch loss around `sympy/integrals/meijerint.py:1736`.

The traced rewrite represents the problem through a Meijer-G expression with argument x^2 and a prefactor $1/x$. In the diagnosed code region, `_meijerint_indefinite_1` extracts powers from the argument and prefactor, substitutes into the Meijer-G antiderivative, applies `hyperexpand`, and then adds a `powdenest(..., polar=True)` simplification:

```
a, b = _get_coeff_exp(g.argument, x)
_, c = _get_coeff_exp(po, x)
...
fac_ = fac * C * x**(1 + c) / b
...
r = hyperexpand(r.subs(t, a*x**b), place=place)
res += powdenest(fac_*r, polar=True)
```

For this integrand, the argument contributes $b = 2$ from x^2 , and the power factor contributes $c = -1$ from $1/x$. The generic substitution–expansion–power–denesting path returns

$$\frac{x}{\sqrt{1 + x^{-2}}} - \text{asinh}\left(\frac{1}{x}\right) + \frac{1}{x\sqrt{1 + x^{-2}}}.$$

That expression is branch-correct only where $\sqrt{1 + x^{-2}}$ denotes the same branch as $\sqrt{x^2 + 1}/x$, namely on the positive real branch. On negative real inputs, the missing $\text{sign}(x)$ factor causes the

derivative to be $+\sqrt{1+x^{-2}}$ instead of $-\sqrt{1+x^{-2}}$. The diagnosis suggests that this Meijer-G path should avoid applying the rewrite globally when an even power substitution such as x^2 is combined with an odd prefactor such as $1/x$, or should instead return a branch-aware result, preserve polar conditions through simplification, or decline to evaluate without suitable assumptions.

1.6 Additional Failing Instances

The independent verification checks the same antiderivative identity at the negative real points $-1, -2, -3, -4, -5, -6$. These are not different formulas; they are pointwise witnesses for the same missing negative-real branch. At every listed point, SymPy's returned antiderivative differentiates to the positive branch $\sqrt{1+n^2}/n$, while the original integrand at $x = -n$ is the corresponding negative value.

Input / Expression	SymPy behavior	Expected behavior
$x = -1$ in $\sqrt{x^2+1}/x$	derivative = $\sqrt{2}$	integrand = $-\sqrt{2}$
$x = -2$ in $\sqrt{x^2+1}/x$	derivative = $\sqrt{5}/2$	integrand = $-\sqrt{5}/2$
$x = -3$ in $\sqrt{x^2+1}/x$	derivative = $\sqrt{10}/3$	integrand = $-\sqrt{10}/3$
$x = -4$ in $\sqrt{x^2+1}/x$	derivative = $\sqrt{17}/4$	integrand = $-\sqrt{17}/4$
$x = -5$ in $\sqrt{x^2+1}/x$	derivative = $\sqrt{26}/5$	integrand = $-\sqrt{26}/5$
$x = -6$ in $\sqrt{x^2+1}/x$	derivative = $\sqrt{37}/6$	integrand = $-\sqrt{37}/6$

The expected behavior is uniform across this family because, for every positive integer n ,

$$\left. \frac{\sqrt{x^2+1}}{x} \right|_{x=-n} = -\frac{\sqrt{n^2+1}}{n}.$$

The observed derivative has the opposite sign at each of these negative-real points.

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The artifact reports the verdict as a new instance of a known failure mode: branch sensitivity in indefinite integration with radicals and inverse hyperbolic rewrites is a broadly known kind of problem, especially around transformations that effectively replace $\sqrt{x^2}$ by x rather than $|x|$. However, no public report was found for this exact integrand, returned antiderivative, or pointwise derivative failure at $x = -2$. The available evidence therefore supports treating this as an individually actionable SymPy issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import diff, integrate, simplify, sqrt, symbols

@pytest.mark.parametrize("n", [1, 2, 3, 4, 5, 6])
def test_integrate_sqrt_x_squared_plus_one_over_x_negative_branch(n):
```

```
x = symbols("x")
integrand = sqrt(x**2 + 1)/x
antiderivative = integrate(integrand, x)
assert simplify((diff(antiderivative, x) - integrand).subs(x, -n)) == 0
```